We will start with Image Generation problem today:-

-We will be using what we learnt so far and use them for a different kind of problem.

You may generate image from nothing (or noise), the case when you ask your GPTs to generate something for you.

Alternatively you may start from another image, making it an image to image problem..with UNET kind of architectures.

We will start with the former today:-

Problem Statement 1: We start from noise and generate image. We do tell the network the type of images we want.

What do you need?

a. *The target example set.*
b. *Of course the network you will use*

Example:--

*I have a 128*100*1*1 (batch_size*ch*h*w) input which I generated using torch.rand function*

*After layer 1 of our network: 128*64*4*4… (upsample+conv or convtranspose)*

*After layer 2 of our network: 128*32*16*16*

*After layer 3 of our network: 128*8*64*64*

*After layer 4 of our network: 128*3*64*64 -say this is my generated rgb image with size 64*64*

The above network is called a generator network.

I have a discriminator network

-it takes input image, uses a few conv+bn+relu+maxpool layers to compute features, and then a sigmoid function to classify the features into real and fake.

DCGAN Tutorial — PyTorch Tutorials 2.9.0+cu128 documentation

Problem Statement 2: : Image to Image Generation using Pix2Pix GAN

What changes with our previous implementation of noise 2 image GAN?

a. Generator(G) and Discriminator(D): I can still use the same discriminator model, generator will be a UNet as now I am generating another image from an input image. Previously my input dim was bs*100*1*1, now it is bs*3*224*224.
b. Training procedure: You can use the same training procedure, you can additionally add a loss (mse) between your predicted image and target image
-generated image (F) from generator, pass real images with label 1 to D and update it D(R), pass fake images with label 0 to D and update it D(F), pass fake images with label 1 and update generatorD(F)
c. Input folders: Previously you had a single folder containing all your images (target), no input as input was noise, you can randomly generate it. Now you will have 2 separate folders, one for input, one for target. Also, there is a one-o-one mapping. This means when you train every input black image should look like a particular color image…so supervision is there.

You can train using a small set (2000 images) of black and white to color problem.

GitHub - junyanz/pytorch-CycleGAN-and-pix2pix: Image-to-Image Translation in PyTorch

1611.07004

We have so far checked

- Noise to image generation (DCGAN) not fully supervised , only sample target set used
- Image to Image generation (Pix2Pix GAN) , can use full one-to-one supervision or unsupervised.

Problem Statement 3 : Generation using Diffusion Models

Now we will get introduced to a few concepts for full Generative AI

- Lora adapters (2106.09685)
- Zero convolutions (2302.05543)
- Diffusion model
- Conditioning (image/text) (2302.08453)
- Unsupervised generation (1703.10593)

These concepts will help build our knowledge towards fully understanding Gen AI image generation

GitHub - GaParmar/img2img-turbo: One-step image-to-image with Stable Diffusion turbo: sketch2image, day2night, and more

2403.12036